Persönliche Videospielverwaltung

Projektbeschreibung

Mein Projekt ist eine Desktop-Anwendung, mit der man seine persönliche Videospielsammlung verwalten kann. Hier können Nutzer ihre Spiele eintragen, deren Status bearbeiten und einsehen (in Besitz, auf der Wunschliste, verkauft, etc.). Ebenso gibt es die Möglichkeit, Spiele zu bewerten und Sammlungen mit mehreren Inhabern anzulegen.

- **Hauptaufgabe**: Zentrale und kompakte Übersicht über die eigenen Spiele und deren Status.
- **Zielgruppe**: Privatpersonen mit großer und kleiner Spielesammlung.
- **Motivation**: Strukturierte Übersicht über die eigene Sammlung, Spiele auf der Wishlist und verkaufte/besessene Spiele mit personalisierten Statistiken, sowie keine Gefahr des doppelten Kaufens bei großen Sammlungen.

Geplante Funktionalitäten

- **Must Have**:
  - Spiele hinzufügen, bearbeiten und entfernen
  - Statuswechsel (Auf der Wunschliste → gekauft/in Besitz → durchgespielt → besessen/verkauft)
  - Sammlung mit mehreren Personen anlegen (z.B.: Familiensammlung)
  - Sammlung anzeigen und nach Kategorien filtern (Genre, Platform, Releasedate, etc.)
- **Should Have**:
  - Persönliche Bewertung mit kurzer Notiz/Kritik
  - Suchfunktion (Titel, Platform, Entwickler, etc.)
- **Nice to Have**:
  - Statistiken (Wieviele Spiele, Gesamtwert, wieviele duchgespielt, etc.)

Bereits implementierten Funktionalitäten

Grundstruktur

- Folgende Klassen implementiert:
  - Game, Platform, Genre, Developer, ReleaseDate, Status, Review, Player, Wishlist und Price
- Getter- und Settermethoden für alle Klassen implementiert
- toString(), equals() und hashCode() implementiert
- Id-Attribute mit final versehen
- Zweiter Konstruktor in Game.java für WishlistHandler.java angelegt
- Zweiten Konstruktor in Review.java für ReviewHandler.java angelegt

Funktionen

- Funktionen addGame() und removeGame() für Wishlist.java implementiert
- GameComparator.java mit Sortierung nach Game, Price, Developer und ReleaseDate erstellt
- Interface SearchCollection für spätere Suchfunktion erstellt

Datenbanken

- SQLite Verbindung nach Anleitungsschritten erstellt und erfolgreich getestet
- MyDatabase.db erstellt
- Handler-Klassen vollständig implementiert
    - Game-Handler
    - Player-Handler
    - Wishlist-Handler
    - Review-Handler
- Hilfsmethode convertToObject() in allen Handler-Klassen implementiert, um Rohdaten aus der Datenbank in ein passendes Objekt zu konvertieren
- 5 Dummy-Daten pro Tabelle in DummyData.java erstellt und erfolgreich getestet

Änderungen seit der letzten Abgabe

Disclaimer

In manchen Klassen werden Logiken und Methoden benutzt, auf die Ich ohne externe Hilfe wie StackOverflow, Reddit und weitere Foren nicht gekommen wäre, wie zum Beispiel LocalDate.MIN oder LocalDate.NOW um Nullcheck sinnvoll zu implementieren. Es wurde **KEINE** KI verwendet und keine Codeblöcke aus Foren Copy-Pasted.

Checkliste von Blatt 2

- **Allgemein**
    - ☑ Überprüfen Sie, ob weiterhin alle Tutor*innen sowie Fatih Özgan Zugang zu Ihrem Repository haben
    - ☑ README.md aktualisiert und als PDF auf Moodle hochgeladen
    - ☑ Feedback zu Blatt 1 und 2 umgesetzt
    - ☑ Ergebnisse auf main-branch gepusht und selbstständig auf der GitLab-Website überprüft
- **Netzwerk**
    - ☑ Client und Server können erfolgreich verbinden
    - ☑ mindestens 3 verschiedene Commands implementiert
    - ☑ Server verarbeitet Anfragen korrekt
    - ☑ Server greift korrekt auf die Datenbank aus Blatt 2 zu
- **DTO / Kommunikation**
    - ☑ Für jede komplexe Anfrage wird eine eigene DTO-Klasse verwendet
    - ☑ keine reine String-basierte Kommunikation
- **Nebenläufigkeit und Synchronisation**
    - ☑ mehrere Clients gleichzeitig getestet
    - ☑ mindestens eine Methode synchronisiert
    - ☑ Race condition getestet und dokumentiert (ganz unten)

Verbesserungen und Überarbeitungen

- weitere packages hinzugefügt(tests, security, dto-packages)
- in Player.java password Attribut, get- und setPassword hinzugefügt
- in PlayerHandler.java password TEXT Spalte in createTable() hinzugefügt
- in Developer.java, Genre.java, Platform.java und Game.java jeweils implements Serializable hinzugefügt

- in GameHandler.java, WishlistHandler.java, ReviewHandler.java und PlayerHandler.java schreibende Methoden mit **synchronized** versehen
    - hierbei habe ich nur "schreibende" methoden mit synchronized versehen, da nur diese aktiv auf die Datenbank zugreifen und Daten verändern können
- in DummyData werden password jetzt korrekt gehasht gespeichert

Neue Klassen

- Commands.java als ENUM mit allen Client Anfragen
- StatusCommand.java als ENUM mit allen Server Antworten
- Server.java um Server auf Port 8080 zu starten
- ClientHandler.java Client in einzelnen Thread zu verarbeiten
- GameClient.java stellt Anfragen an Server
- DummyClients.java startet zwei GameClients testweise gleichzeitig

Neue Funktionen

- in **GameClient.java**: login(), registration(), getGames(), addGames(), deleteGames(), getWishlist(), addToWishlist(), updateWishlist(), addReview(), start()
- in **LoginHandler**: hashPW(), registration(), login()

Datenbankstruktur

- Folgende Tabellen wurden erstellt:
    1. games mit Primary Key gameId → Speichert alle übergebenen Spielinformationen
    2. players mit Primary Key playerId → speichert das Nutzerprofil
    3. wishlist mit Primary Key playerId und gameId → ordnet jedem Nutzer eine Wishlist zu und speichert sie
    4. reviews mit Primary Key reviewId → Speichert Nutzerreviews

Noch zu implementieren

- Statistics.java → Wieviele Spiele, Gesamtwert der Sammlung anhand des Einkaufpreises, etc
- GUI

Umsetzung von Feedback zu Blatt 2

- "Zweitkonstruktoren in Game sind inkonsistent in Parameterreihenfolge" wurde gefixt
- "DatabaseConnection.connect() gibt bei Fehler null zurück" → habe **throws SQLException** hinzugefügt
- "Datumskonvertierung sollte auf null/Formatfehler geprüft werden" → habe Nullcheck in der convertToObject()-Methode in GameHandler.java implementiert, sowie Nullcheck im Zweitkonstruktor von Review. LocalDate.MIN und LocalDate.NOW() dienen als Absicherung, damit trotzdem ein Wert übergeben wird

Race Condition Dokumentation

Ausgangssituation ist, dass zwei Clients gleichzeitig auf dieselbe Ressource zugreifen und versuchen, ein Spiel mit gameId 1234 zu löschen. Ohne Synchronisation würden beide Threads gleichzeitig auf die Datenbank zugreifen, was zu Datenbankfehlern führt. Mit Synchronisation kann hier GameHandler sicherstellen, dass

immer nur genau ein Thread gleichzeitig auf die Datenbank zugreifen kann. Der zweite Thread würde in unserem Fall dann warten bis der erste fertig ist. Ich habe mich für einen TCP Socket entschieden, da beim Übertragen der Login-Daten die Zuverlässigkeit sehr wichtig ist. Ebenso kann Datenverlust fatal sein und die Reihenfolge der gesendeten Pakete muss garantiert sein.